GitHub + Railway Deployment Cheatsheet

Personal reference from the task-manager project (May 2026).

One-time setup (new project)

Step 1 — Initialize git in your project folder

Open Terminal, navigate to your project root, and run `git init`. This creates a hidden `.git` folder that starts tracking your project.

```
cd /Users/manousarr/Dokument/Claude/Projects/mdfiletest
git init
# Output: Initialized empty Git repository in .../mdfiletest/.git/
```

Step 2 — Create a .gitignore file

This file tells git which files to never upload. Critical: `node_modules` is huge (thousands of files) and `.env` contains your secrets.

Create a file called `.gitignore` at the root of your project with this content:

```
# dependencies
node_modules/

# environment variables — NEVER commit this
.env

# macOS
.DS_Store

# logs
*.log
```

In the task-manager project this file lives at:

```
/Users/manousarr/Dokument/Claude/Projects/mdfiletest/.gitignore
```

Step 3 — Stage all files

"Staging" means telling git which files to include in the next snapshot. The `.` means "everything in this folder" — your `.gitignore` will automatically exclude the protected files.

```
git add .

# Verify what will be committed (should NOT show .env or node_modules)
git status
# Output:
# Changes to be committed:
#   new file: .gitignore
#   new file: package.json
#   new file: src/index.js
#   new file: public/index.html
#   ... etc
```

Step 4 — Make your first commit

A commit is a permanent snapshot of your code with a description of what it contains.

```
git commit -m "Initial commit — task manager with auth, MongoDB, and
redesigned UI"
# Output:
# [main (root-commit) a8c8dba] Initial commit — task manager with auth...
#  18 files changed, 7571 insertions(+)
```

Step 5 — Create an empty repo on GitHub

1. Go to **github.com** and sign in
2. Click **+** in the top-right corner → **"New repository"**
3. Fill in:
   - Repository name: e.g. `task-manager`
   - Keep it **Public**
   - Do **NOT** check "Add a README" or "Add .gitignore" — you already have your own
4. Click **"Create repository"**
5. Copy the URL GitHub shows you, e.g.: `https://github.com/VibeCoder707/task-manager.git`

Step 6 — Link your local project to GitHub

```
git remote add origin https://github.com/VibeCoder707/task-manager.git
```

`remote add origin` means: "the remote destination named `origin` is at this URL"

Step 7 — Push to GitHub

```
git push -u origin main
# Output:
# * [new branch] main -> main
# branch 'main' set up to track 'origin/main'
```

The `-u origin main` is only needed the first time. It sets the default so future pushes are just `git push`.

Set up Railway (one-time, after GitHub is set up)

1. Go to **railway.app** → Login with GitHub
2. **New Project** → **GitHub Repository** → select your repo (e.g. `task-manager`)
3. Railway starts building — it will crash initially because env vars are missing
4. Click the service box → **Variables tab** → add your secrets:

Name	Value
`MONGO_URI`	your `mongodb+srv://...` connection string from MongoDB Atlas
`JWT_SECRET`	your long random secret string
`NODE_ENV`	`production`

5. Click the purple **Deploy** button to redeploy with the variables applied
6. Go to **Settings tab** → **Generate Domain**
7. Your app is now live at a URL like: `something.up.railway.app`

Everyday workflow (after setup)

Every time you make changes and want them live, run these 3 commands:

```
# 1. Stage your changes
git add .

# 2. Commit with a descriptive message
git commit -m "Describe what you changed"
# Example: git commit -m "Personalise app name, accent color, and empty
state"

# 3. Push to GitHub
git push
```

Railway detects the push within seconds and automatically redeploys.

What happens automatically after git push

```
git push
  │
  ▼
GitHub receives the new code
  │
  ▼
GitHub Actions runs your tests automatically
  │
  ├── ✅ Tests pass → Railway deploys → live site updates
  │
  └── ❌ Tests fail → Railway is NOT updated
                    → red X on GitHub Actions tab
                    → fix the bug, then push again
```

Check CI results any time at:

```
github.com/VibeCoder707/task-manager/actions
```

The CI workflow file that makes this happen lives at:

```
.github/workflows/ci.yml
```

Handy git commands

```
git status          # see which files changed since last commit
git log --oneline   # see full commit history, one line per commit
git diff            # see exactly which lines changed
```

The mental model

Tool	What it does
Git	Save system for code — every commit is a snapshot you can return to
GitHub	Cloud backup of all commits — visible to collaborators and services
GitHub Actions	Runs your tests automatically on every push
Railway	Watches your GitHub repo and keeps your live app in sync with it

---

This project's live URLs

- **GitHub repo:** https://github.com/VibeCoder707/task-manager
- **Live app:** https://task-manager-production-b3ab.up.railway.app